VIETNAM NATIONAL UNIVERSITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Digital Systems
Lab 6

Instructor: Đoàn Nọc Cẩm

Student	Student ID
Hồ Chí Dũng	2351016
Nguyễn Viết Thi	2451165
Trần Duy Anh	2351007

Laboratory 6:

AN ENHANCED PROCESSOR WITH FLOATING POINT UNIT

OBJECTIVES

- The purpose of this lab is to learn how to connect simple input (switches) and output devices (LEDs and 7-segment) to an FPGA chip and implement a circuit that uses these devices.
- Based on the processor in lab 5, design a processor which can execute both integer number and floating-point number arithmetic operations.

PREPARATION FOR LAB 6

- Finish all the lab from 1-5.
- Students have to do lab 6, then write the report of this project. The report should include all steps of your design: block diagram, waveform, RTL viewer, FSM,…
- When reporting to the instructors, you will answer random questions to prove you understand the project.

REFERENCE

1. Intel FPGA training

Laboratory 6:

AN ENHANCED PROCESSOR WITH FLOATING POINT UNIT

Requirement: Connect floating-point unit (designed in lab 3) into the enhanced processor in lab5.

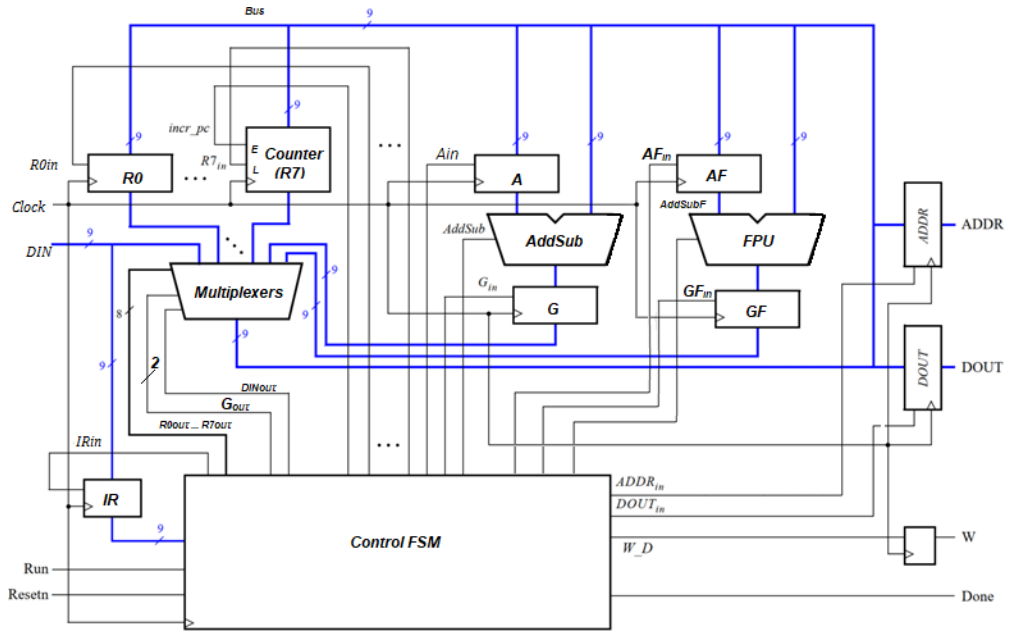


Figure 1: A reference schematic of the enhanced processor with floating point unit

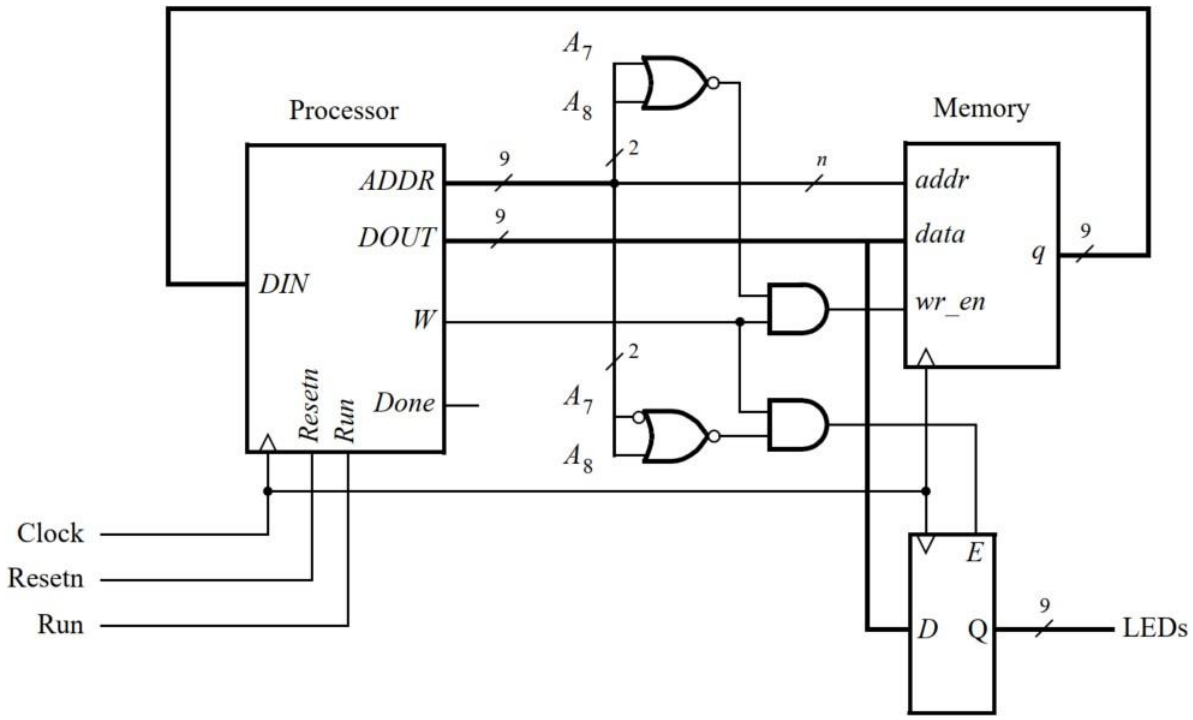


Figure 2: Connecting the enhanced processor to a memory and output register.

Laboratory 6:

AN ENHANCED PROCESSOR WITH FLOATING POINT UNIT

Instruction:

The FPU operate on 9-bit floating-point representations. There is one sign bit, four exponent bits (exponent bias is 7) and four fraction bits. Modify the FPU in lab3 and instance the module on the design.

You are required to add one new instruction which operate with floating-point number, illustrated on Figure 3.

Operation	Function performed
addf Rx, Ry	Rx ← Rx + Ry (data stored in Rx, Ry is in floating-point representation)

Figure 3: New instruction perpormed in the processor.

Similar to **add** and **sub** instruction, it take more than one clock to complete **addf** instruction. The control unit asserts control signals needed in successive clock cycle until the instruction has completed. When the controller detect **addf** 's opcode, it activates FPU block by setting appropriate value of **AFin**, **AddSubF**, **GFin**. In this lab, there is only one opcode left for floating-point arithmetic, hence by default, the FPU execute only the addition operation (i.e AddSubF = 0). The AddSubF signal, however, still be added in the system for later extension.

The selection signal, **Gout** are converted to 2-bit signal that identifies which result is loaded to the **Bus** (Gout = 10: select AddSub output; Gout = 01: select AddSubF output).

After completing the design, compile the circuit, download it into the FPGA chip, and ensure that your program runs properly. (Students need to write some programs that verify the design correctness).

CODE:

//TOP MODULE:

```
module DE10 (
    input  logic       CLOCK_50, // Xung clock 50MHz của mạch
    input  logic [0:0] KEY,      // KEY[0] dùng làm nút bấm Resetn
    input  logic [9:9] SW,       // SW[9] dùng làm công tắc Run
    output logic [8:0] LEDR      // LEDR[8:0] nối ra 9 bóng đèn LED đỏ
);

    // --- 1. KHAI BÁO DÂY NỐI NỘI BỘ ---
    logic [8:0] ADDR;
    logic [8:0] DOUT;
    logic [8:0] DIN;
    logic       W;
    logic       Done;
    logic [8:0] BusWires; // Giữ lại để debug nếu cần

    // Tín hiệu giải mã địa chỉ (Address Decoding)
    logic ram_wr_en;
    logic led_en;

    // RAM được ghi khi 2 bit địa chỉ cao nhất A8, A7 = 00 VÀ có cờ Write (W=1)
       assign ram_wr_en = ( ~(ADDR[8] | ADDR[7]) ) & W;

    // LED được cập nhật khi A8, A7 = 01 VÀ có cờ Write (W=1)
       assign led_en = ( ~(ADDR[8] | ~ADDR[7]) ) & W;

    // --- 3. CẮM CÁC THÀNH PHẦN VÀO HỆ THỐNG ---

    // A. Lõi Vi xử lý (Processor) - ĐÃ ĐỔI TÊN THÀNH lab6
    lab6 cpu (
        .DIN(DIN),
        .Resetn(KEY[0]),
        .Clock(CLOCK_50),
        .Run(SW[9]),
        .Done(Done),
        .BusWires(BusWires),
        .ADDR(ADDR),
        .DOUT(DOUT),
        .W(W)
    );

    // B. Bộ nhớ RAM (Memory 128 words x 9 bits)
    // Lưu ý: Module này KHÔNG tự có sẵn, bạn phải tạo bằng Quartus IP Catalog!
    RAM my_ram (
        .address (ADDR[6:0]),  // Chỉ lấy 7 bit thấp (A6 -> A0) để quét 128 ô
nhớ
        .clock   (CLOCK_50),   // RAM đồng bộ dùng chung xung clock
        .data    (DOUT),       // Dữ liệu từ CPU đẩy ra chờ ghi vào RAM
        .wren    (ram_wr_en),  // Cờ báo hiệu ghi vào RAM
        .q       (DIN)         // Dữ liệu RAM nhả ra để CPU đọc (lệnh hoặc data)
    );
```

```
    // C. Thanh ghi LED (Output Register)
    always_ff @(posedge CLOCK_50 or negedge KEY[0]) begin
        if (!KEY[0]) begin
            LEDR <= 9'b000000000; // Reset tắt hết bóng đèn
        end else if (led_en) begin
            LEDR <= DOUT;         // Chốt dữ liệu từ DOUT ra đèn LED
        end
    end

endmodule

//LAB 6
//
// ============================================================================
// MODULE 1: PROCESSOR
//
// ============================================================================
module lab6 #(parameter n = 9)(
    input  logic [8:0] DIN,
    input  logic       Resetn, Clock, Run,
    output logic       Done,
    output logic [8:0] BusWires, // Giữ lại để xem waveform
    output logic [n-1:0] ADDR,
    output logic [n-1:0] DOUT,
    output logic       W
);
    // ----------------------------------------------------------------------
-
    // Khai báo dây nối nội bộ
    // ----------------------------------------------------------------------
-
    logic [8:0] IR;
    logic [8:0] R0, R1, R2, R3, R4, R5, R6, R7, A, G;
    logic [8:0] AddSub_Result;
    logic       IRin, DINout, Ain, Gin, AddSub;
    logic [7:0] Rin, Rout;
    logic       ADDRin, DOUTin, W_D, incr_pc;
    logic       G_not_zero;

    // FPU Signals
    logic [8:0] AF, GF;              // Thanh ghi cho FPU
    logic       AFin, GFin, AddSubF; // Tín hiệu điều khiển FPU
    logic [1:0] Gout;                // Nâng cấp Gout lên 2-bit
    logic [8:0] AddSub_Result_F;

    // ----------------------------------------------------------------------
-
    // Logic phụ trợ
    // ----------------------------------------------------------------------
-
    // Kiểm tra G != 0 cho lệnh mvnz
    assign G_not_zero = (G != 9'b000000000);

    // Mạch đồng bộ 2 tầng cho chân Run (Chống Metastability)
```

```
    logic run_sync_1, run_sync_2;
    always_ff @(posedge Clock or negedge Resetn) begin
        if (!Resetn) begin
            run_sync_1 <= 1'b0;
            run_sync_2 <= 1'b0;
        end else begin
            run_sync_1 <= Run;
            run_sync_2 <= run_sync_1;
        end
    end

    // Flip-flop W: Chốt cờ Write (1 bit)
    always_ff @(posedge Clock or negedge Resetn) begin
        if (!Resetn)
            W <= 1'b0;
        else
            W <= W_D;
    end

    // ------------------------------------------------------------------------
-
    // Kết nối các khối (Instantiations)
    // ------------------------------------------------------------------------
-
    // 1. Khối Điều khiển (Control Unit)
    control_unit CU (
        .Clock(Clock),
        .Resetn(Resetn),
        .Run(run_sync_2),
        .IR(IR),
        .G_not_zero(G_not_zero),
        .IRin(IRin),
        .DINout(DINout),
        .Ain(Ain),
        .Gin(Gin),
        .AFin(AFin),
        .GFin(GFin),
        .Gout(Gout),
        .AddSub(AddSub),
        .AddSubF(AddSubF),
        .Done(Done),
        .Rin(Rin),
        .Rout(Rout),
        .ADDRin(ADDRin),
        .DOUTin(DOUTin),
        .W_D(W_D),
        .incr_pc(incr_pc)
    );

    // 2. Các thanh ghi cơ bản & Memory
    regn reg_IR   (.R(DIN),      .Rin(IRin),   .Resetn(Resetn), .Clock(Clock),
.Q(IR));
    regn reg_ADDR (.R(BusWires), .Rin(ADDRin), .Resetn(Resetn), .Clock(Clock),
.Q(ADDR));
```

```
    regn reg_DOUT (.R(BusWires), .Rin(DOUTin), .Resetn(Resetn), .Clock(Clock),
.Q(DOUT));

    // 3. Thanh ghi đa dụng (R0 - R6)
    regn reg_0 (.R(BusWires), .Rin(Rin[0]), .Resetn(Resetn), .Clock(Clock),
.Q(R0));
    regn reg_1 (.R(BusWires), .Rin(Rin[1]), .Resetn(Resetn), .Clock(Clock),
.Q(R1));
    regn reg_2 (.R(BusWires), .Rin(Rin[2]), .Resetn(Resetn), .Clock(Clock),
.Q(R2));
    regn reg_3 (.R(BusWires), .Rin(Rin[3]), .Resetn(Resetn), .Clock(Clock),
.Q(R3));
    regn reg_4 (.R(BusWires), .Rin(Rin[4]), .Resetn(Resetn), .Clock(Clock),
.Q(R4));
    regn reg_5 (.R(BusWires), .Rin(Rin[5]), .Resetn(Resetn), .Clock(Clock),
.Q(R5));
    regn reg_6 (.R(BusWires), .Rin(Rin[6]), .Resetn(Resetn), .Clock(Clock),
.Q(R6));

    // Program Counter (R7)
    pc_register pc_reg_7 (
        .R(BusWires),
        .R7in(Rin[7]),
        .incr_pc(incr_pc),
        .Clock(Clock),
        .Resetn(Resetn),
        .Q(R7)
    );

    // 4. Thanh ghi cho ALU & FPU
    regn reg_A  (.R(BusWires),      .Rin(Ain),  .Resetn(Resetn), .Clock(Clock),
.Q(A));
    regn reg_G  (.R(AddSub_Result), .Rin(Gin),  .Resetn(Resetn), .Clock(Clock),
.Q(G));
    regn reg_AF (.R(BusWires),      .Rin(AFin), .Resetn(Resetn), .Clock(Clock),
.Q(AF));
    regn reg_GF (.R(AddSub_Result_F), .Rin(GFin), .Resetn(Resetn),
.Clock(Clock), .Q(GF));

    // 5. Khối tính toán (ALU & FPU)
    alu ALU_unit (
        .A(A), .BusWires(BusWires), .AddSub(AddSub), .Result(AddSub_Result)
    );

    lab3_r FPU_unit (
        .A(AF),
        .B(BusWires),
        .S(AddSubF),
        .Result(AddSub_Result_F),
        .z(),
        .OV()
    );

    // 6. Bộ Dồn kênh (Multiplexer)
```

```
    multiplexer MUX_unit (
        .R0(R0), .R1(R1), .R2(R2), .R3(R3), .R4(R4), .R5(R5), .R6(R6), .R7(R7),
        .G(G), .GF(GF), .DIN(DIN),
        .Rout(Rout), .DINout(DINout), .Gout(Gout),
        .BusWires(BusWires)
    );

endmodule

//
================================================================================
// MODULE 2: PROGRAM COUNTER (R7)
//
================================================================================
module pc_register #(parameter n = 9)(
    input  logic [n-1:0] R,
    input  logic         R7in,
    input  logic         incr_pc,
    input  logic         Clock, Resetn,
    output logic [n-1:0] Q
);
    always_ff @(posedge Clock or negedge Resetn) begin
        if (!Resetn)
            Q <= '0;
        else if (R7in)
            Q <= R;
        else if (incr_pc)
            Q <= Q + 1'b1;
    end
endmodule

//
================================================================================
// MODULE 3: DECODER 3-TO-8
//
================================================================================
module dec3to8(
    input  logic [2:0] W,
    input  logic       En,
    output logic [7:0] Y
);
    always_comb begin
        if (En == 1'b1)
            case (W)
                3'b000: Y = 8'b00000001;
                3'b001: Y = 8'b00000010;
                3'b010: Y = 8'b00000100;
                3'b011: Y = 8'b00001000;
                3'b100: Y = 8'b00010000;
                3'b101: Y = 8'b00100000;
                3'b110: Y = 8'b01000000;
                3'b111: Y = 8'b10000000;
            endcase
        else
```

```
            Y = 8'b00000000;
    end
endmodule

//
================================================================================
// MODULE 4: N-BIT REGISTER
//
================================================================================
module regn #(parameter n = 9)(
    input  logic [n-1:0] R,
    input  logic         Rin, Clock, Resetn,
    output logic [n-1:0] Q
);
    always_ff @(posedge Clock or negedge Resetn) begin
        if (!Resetn)
            Q <= '0;
        else if (Rin)
            Q <= R;
    end
endmodule

//
================================================================================
// MODULE 5: ALU
//
================================================================================
module alu (
    input  logic [8:0] A,
    input  logic [8:0] BusWires,
    input  logic       AddSub,
    output logic [8:0] Result
);
    always_comb begin
        if (AddSub)
            Result = A - BusWires;
        else
            Result = A + BusWires;
    end
endmodule

//
================================================================================
// MODULE 6: MULTIPLEXER
//
================================================================================
module multiplexer (
    input  logic [8:0] R0, R1, R2, R3, R4, R5, R6, R7,
    input  logic [8:0] G, GF, DIN,
    input  logic [7:0] Rout,
    input  logic       DINout,
    input  logic [1:0] Gout,
    output logic [8:0] BusWires
);
```

```
    always_comb begin
        BusWires = 9'b000000000; // Mặc định

        if (DINout)              BusWires = DIN;
        else if (Gout == 2'b10)  BusWires = G;
        else if (Gout == 2'b01)  BusWires = GF;
        else if (Rout[0])        BusWires = R0;
        else if (Rout[1])        BusWires = R1;
        else if (Rout[2])        BusWires = R2;
        else if (Rout[3])        BusWires = R3;
        else if (Rout[4])        BusWires = R4;
        else if (Rout[5])        BusWires = R5;
        else if (Rout[6])        BusWires = R6;
        else if (Rout[7])        BusWires = R7;
    end
endmodule

//
=============================================================================
// MODULE 7: CONTROL UNIT (FSM)
//
=============================================================================
module control_unit (
    input  logic       Clock, Resetn, Run,
    input  logic [8:0] IR,
    input  logic       G_not_zero,
    output logic       IRin, DINout, Ain, Gin, GFin, AFin, AddSub, AddSubF,
Done,
    output logic [1:0] Gout,
    output logic [7:0] Rin, Rout,
    output logic       ADDRin, DOUTin, W_D, incr_pc
);
    typedef enum logic [2:0] {T0=3'd0, T1=3'd1, T2=3'd2, T3=3'd3, T4=3'd4,
T5=3'd5, T6=3'd6} state_t;
    state_t Tstep_Q, Tstep_D;

    // Phân rã mã lệnh
    logic [2:0] I;
    logic [7:0] Xreg, Yreg;
    assign I = IR[8:6];

    dec3to8 decX (.W(IR[5:3]), .En(1'b1), .Y(Xreg));
    dec3to8 decY (.W(IR[2:0]), .En(1'b1), .Y(Yreg));

    // FSM State Register
    always_ff @(posedge Clock, negedge Resetn) begin
        if (!Resetn) Tstep_Q <= T0;
        else         Tstep_Q <= Tstep_D;
    end

    // FSM Next State Logic
    always_comb begin
        case (Tstep_Q)
            T0: if (!Run) Tstep_D = T0; else Tstep_D = T1;
```

```
            T1: Tstep_D = T2;
            T2: Tstep_D = T3;
            T3: if (Done) Tstep_D = T0; else Tstep_D = T4;
            T4: if (Done) Tstep_D = T0; else Tstep_D = T5;
            T5: if (Done) Tstep_D = T0; else Tstep_D = T6;
            T6: Tstep_D = T0;
            default: Tstep_D = T0;
        endcase
    end

    // FSM Output Logic
    always_comb begin
        // Khởi tạo mặc định
        IRin = 0; DINout = 0; Ain = 0; Gin = 0; Gout = 2'b00;
        AFin = 0; GFin = 0; AddSub = 0; AddSubF = 0;
        Rin = 8'b0; Rout = 8'b0; Done = 0;
        ADDRin = 0; DOUTin = 0; W_D = 0; incr_pc = 0;

        case (Tstep_Q)
            T0: begin
                Rout[7] = 1'b1;
                ADDRin  = 1'b1;
            end
            T1: begin
                // NHỊP CHỜ RAM
            end
            T2: begin
                IRin    = 1'b1;
                incr_pc = 1'b1;
            end

            T3: begin
                case (I)
                    3'b000: begin Rout = Yreg; Rin = Xreg; Done = 1; end
                    3'b001: begin Rout[7] = 1'b1; ADDRin = 1'b1; end
                    3'b010,
                    3'b011: begin Rout = Xreg; Ain = 1; end
                    3'b111: begin Rout = Xreg; AFin = 1; end
                    3'b100: begin Rout = Yreg; ADDRin = 1'b1; end
                    3'b101: begin Rout = Yreg; ADDRin = 1'b1; end
                    3'b110: begin
                        if (G_not_zero) begin
                            Rout = Yreg;
                            Rin = Xreg;
                        end
                        Done = 1;
                    end
                endcase
            end

            T4: begin
                case (I)
                    3'b001: ;
                    3'b010: begin Rout = Yreg; Gin = 1; AddSub = 0; end
```

```
                    3'b011: begin Rout = Yreg; Gin = 1; AddSub = 1; end
                    3'b111: begin Rout = Yreg; GFin = 1; AddSubF = 0; end
                    3'b100: ;
                    3'b101: begin Rout = Xreg; DOUTin = 1'b1; end
                endcase
            end

            T5: begin
                case (I)
                    3'b001: begin DINout = 1; Rin = Xreg; incr_pc = 1; Done = 1;
end
                    3'b010,
                    3'b011: begin Gout = 2'b10; Rin = Xreg; Done = 1; end
                    3'b111: begin Gout = 2'b01; Rin = Xreg; Done = 1; end
                    3'b100: begin DINout = 1; Rin = Xreg; Done = 1; end
                    3'b101: begin W_D = 1'b1; end
                endcase
            end

            T6: begin
                case (I)
                    3'b101: begin Done = 1; end
                endcase
            end
        endcase
    end
endmodule
```

//LAB 3

```
module lab3_r(
      input logic [8:0] A,
      input logic [8:0] B,
      input logic             S,
      output logic [8:0] Result,
      output logic z,
      output logic OV
);
      //Tách làm 3 phần là sign, frac, exp
      logic sign_A;
      logic [3:0] exp_A;
      logic [3:0] frac_A;

      logic sign_B;
      logic [3:0] exp_B;
      logic [3:0] frac_B;

      assign sign_A = A[8];
      assign sign_B = B[8];
      assign exp_A  = A[7:4];
      assign exp_B  = B[7:4];
      assign frac_A = A[3:0];
      assign frac_B = B[3:0];

      // --- BƯỚC 2: Xử lý phép Trừ ---
```

```
    // Nếu S=1 (Trừ), đảo dấu của B. Nếu S=0 (Cộng), giữ nguyên dấu của B.
   logic eff_sign_B;
   assign eff_sign_B = sign_B ^ S;

      logic [4:0] w_exp_diff; //Thêm 1 bit
      // logic [2:0] w_Smaller_exp;
   logic [3:0] w_final_exp_temp;//Thêm 1 bit
   logic [3:0] w_bigger_frac;
   logic [3:0] w_smaller_frac;
   logic [5:0] w_shifted_frac;
   logic [3:0] w_result_frac;
   logic [3:0] w_result_exp;//Thêm 1 bit
   logic       w_result_sign;

      prelab3_tn1 u1(
            .Exp_A(exp_A),
            .Exp_B(exp_B),
            .Exp_diff(w_exp_diff),
            // .Smaller_exp(w_Smaller_exp),
            .Final_exp(w_final_exp_temp)
);
      prelab3_tn3 u3(
            .Fract_A(frac_A),
            .Fract_B(frac_B),
            .Exp_diff(w_exp_diff),
            .Bigger_fract(w_bigger_frac),
            .Smaller_fract(w_smaller_frac)
);

      prelab3_tn2 u2 (
       .Smaller_fract(w_smaller_frac),
             //.Smaller_exp(w_Smaller_exp),
       .Exp_diff(w_exp_diff),
       .Shifted_fract(w_shifted_frac)
);
      //    Put the + - of the S into the sign of the B
      logic  eff_sign;
      assign eff_sign = sign_A ^ eff_sign_B;

      //Extent the bigger_frac to 6 bit. to match with the shifter_frac
      logic  [5:0] bigger_frac_6b;
      assign bigger_frac_6b = {1'b0, 1'b1, w_bigger_frac};

      logic [5:0] w_Temp_fract;
      assign w_Temp_fract = (eff_sign == 1'b1)? (bigger_frac_6b -
w_shifted_frac) : (bigger_frac_6b + w_shifted_frac);

      prelab3_tn4 u4(
            .Temp_fract(w_Temp_fract),
            .Final_exp(w_final_exp_temp),
            .Result_fract(w_result_frac),
            .Result_exp(w_result_exp)
      );
```

```
	prelab3_tn5 u5(
		.Exp_diff(w_exp_diff),
		.Fract_A(frac_A),
		.Fract_B(frac_B),
		.sign_A(sign_A),
		.sign_B(eff_sign_B),
		.Result_sign(w_result_sign)
	);

	//Assign the 0 result
	assign z = (w_Temp_fract == 6'b000000) ? 1'b1: 1'b0;

	//Assign the overflow result
	assign OV = (w_final_exp_temp == 3'd7) || (w_final_exp_temp == 3'd6 &&
w_Temp_fract[5] == 1'b1) ? 1'b1 : 1'b0;

	assign Result = (z == 1'b1) ? 8'b00000000 : {w_result_sign, w_result_exp,
w_result_frac};

endmodule

module prelab3_tn1(
	input logic [3:0] Exp_A,
	input logic [3:0] Exp_B,
	output logic [4:0] Exp_diff,
	output logic [3:0] Final_exp
);

assign Exp_diff = {1'b0, Exp_A} - {1'b0, Exp_B};

assign Final_exp = (Exp_diff[4] == 1'b1) ? Exp_B: Exp_A;

endmodule

module prelab3_tn2(
	input logic  [3:0] Smaller_fract,
	input logic  [4:0] Exp_diff,
	output logic [5:0] Shifted_fract
);
//Because exp_diff can be positive or negative ==> change to positive

logic [4:0] shifted_val;

always_comb begin
	shifted_val = (Exp_diff[4] == 1)? - Exp_diff: Exp_diff;
	case (shifted_val)
		5'd0: Shifted_fract = {1'b0, 1'b1, Smaller_fract};
		5'd1: Shifted_fract = {2'b00, 1'b1, Smaller_fract[3:1]};
		5'd2: Shifted_fract = {3'b000, 1'b1, Smaller_fract[3:2]};
		5'd3: Shifted_fract = {4'b0000, 1'b1, Smaller_fract[3:3]};
		5'd4: Shifted_fract = {5'b00000, 1'b1};
		default: Shifted_fract = {6'b000000};
	endcase
end
```

```
endmodule

module prelab3_tn3(
      input logic [3:0] Fract_A,
      input logic [3:0] Fract_B,
      input logic [4:0] Exp_diff,
      output logic [3:0] Bigger_fract,
      output logic [3:0] Smaller_fract
);
always_comb begin
      if (Exp_diff == 5'b0000) begin
            if(Fract_A >= Fract_B) begin
                  Bigger_fract = Fract_A;
                  Smaller_fract = Fract_B;
            end
            else begin
                  Bigger_fract = Fract_B;
                  Smaller_fract = Fract_A;
            end
      end
      else begin
            if(Exp_diff[4] == 1'b1) begin
                        Bigger_fract = Fract_B;
                        Smaller_fract = Fract_A;
            end
            else begin
                        Bigger_fract = Fract_A;
                        Smaller_fract = Fract_B;
            end
      end
end
endmodule

module prelab3_tn4 (
    input  logic [5:0] Temp_fract,  // 6-bit intermediate fraction
    input  logic [3:0] Final_exp,   // Original exponent from larger operand
    output logic [3:0] Result_fract, // 4-bit normalized fraction
    output logic [3:0] Result_exp    // 4-bit normalized exponent
);

    // Use a priority logic to find the leading '1'
    // This structure helps RTL Viewer create a cleaner Encoder block
    always_comb begin
        if (Temp_fract[5]) begin
            // CASE: Overflow (Carry-out) -> Shift right, Exponent + 1
            Result_fract = Temp_fract[4:1];
            Result_exp   = Final_exp + 4'd1;
        end
        else if (Temp_fract[4]) begin
            // CASE: Already Normalized -> No shift, Exponent stays same
            Result_fract = Temp_fract[3:0];
            Result_exp   = Final_exp;
        end
```

```
        else if (Temp_fract[3]) begin
            // CASE: Shift left 1, Exponent - 1
            Result_fract = {Temp_fract[2:0], 1'b0};
            Result_exp   = Final_exp - 4'd1;
        end
        else if (Temp_fract[2]) begin
            // CASE: Shift left 2, Exponent - 2
            Result_fract = {Temp_fract[1:0], 2'b00};
            Result_exp   = Final_exp - 4'd2;
        end
        else if (Temp_fract[1]) begin
            // CASE: Shift left 3, Exponent - 3
            Result_fract = {Temp_fract[0], 3'b000};
            Result_exp   = Final_exp - 4'd3;
        end
        else if (Temp_fract[0]) begin
            // CASE: Shift left 4, Result fraction becomes 0
            Result_fract = 4'b0000;
            Result_exp   = Final_exp - 4'd4;
        end
        else begin
            // CASE: Default/Zero -> Reset output
            Result_fract = 4'b0000;
            Result_exp   = 4'd0;
        end
    end

endmodule

module prelab3_tn5(
      input logic [4:0] Exp_diff,
      input logic [3:0] Fract_A,
      input logic [3:0] Fract_B,
      input logic  sign_A,
      input logic  sign_B,
      output logic Result_sign
);

always_comb begin
      if (Exp_diff[4] == 1'b1) begin
            Result_sign = sign_B;
      end
      else if(Exp_diff != 5'b0000) begin
            Result_sign = sign_A;
      end
      else begin
            if(Fract_B > Fract_A) begin
                  Result_sign = sign_B;
            end
            else begin
                  Result_sign = sign_A;
            end
      end
end
```

```
endmodule


// --- Bước 1: Khối D-Flip-Flop cơ bản (Giữ nguyên) ---
module dff_ff (
    input  logic d,
    input  logic clk,
    input  logic rst, // Reset bất đồng bộ
    input  logic en,  // Enable đồng bộ
    output logic q
);
    always_ff @(posedge clk or posedge rst) begin
        if (rst)
            q <= 1'b0;
        else if (en)
            q <= d;
    end
endmodule


// --- Bước 2: Register N-bit sử dụng vòng lặp Generate ---
module register #(parameter N = 8) (
    input  logic [N-1:0] D,
    input  logic         clk,
    input  logic         RST,
    input  logic         EN,
    output logic [N-1:0] Q
);
    // Khai báo biến genvar để chạy vòng lặp sinh phần cứng
    genvar i;
    // Khối generate để tự động nhân bản D-Flip-Flop
    generate
        for (i = 0; i < N; i = i + 1) begin : gen_dff
            // Tạo ra N khối dff_ff, mỗi khối xử lý bit thứ i
            dff_ff dff_inst (
                .d(D[i]),
                .clk(clk),
                .rst(RST),
                .en(EN),
                .q(Q[i])
            );
        end
    endgenerate

endmodule
```

// RAM USING IP CATALOG

```
// synopsys translate_off
`timescale 1 ps / 1 ps
// synopsys translate_on
module RAM (
      address,
      clock,
```

```
      data,
      wren,
      q);

      input [6:0]  address;
      input   clock;
      input [8:0]  data;
      input   wren;
      output      [8:0]  q;
`ifndef ALTERA_RESERVED_QIS
// synopsys translate_off
`endif
      tri1    clock;
`ifndef ALTERA_RESERVED_QIS
// synopsys translate_on
`endif

      wire [8:0] sub_wire0;
      wire [8:0] q = sub_wire0[8:0];

      altsyncram  altsyncram_component (
                  .address_a (address),
                  .clock0 (clock),
                  .data_a (data),
                  .wren_a (wren),
                  .q_a (sub_wire0),
                  .aclr0 (1'b0),
                  .aclr1 (1'b0),
                  .address_b (1'b1),
                  .addressstall_a (1'b0),
                  .addressstall_b (1'b0),
                  .byteena_a (1'b1),
                  .byteena_b (1'b1),
                  .clock1 (1'b1),
                  .clocken0 (1'b1),
                  .clocken1 (1'b1),
                  .clocken2 (1'b1),
                  .clocken3 (1'b1),
                  .data_b (1'b1),
                  .eccstatus (),
                  .q_b (),
                  .rden_a (1'b1),
                  .rden_b (1'b1),
                  .wren_b (1'b0));
      defparam
            altsyncram_component.clock_enable_input_a = "BYPASS",
            altsyncram_component.clock_enable_output_a = "BYPASS",
            altsyncram_component.init_file = "DE10.mif",
            altsyncram_component.intended_device_family = "Cyclone V",
            altsyncram_component.lpm_hint = "ENABLE_RUNTIME_MOD=NO",
            altsyncram_component.lpm_type = "altsyncram",
            altsyncram_component.numwords_a = 128,
            altsyncram_component.operation_mode = "SINGLE_PORT",
            altsyncram_component.outdata_aclr_a = "NONE",
```

```
            altsyncram_component.outdata_reg_a = "UNREGISTERED",
            altsyncram_component.power_up_uninitialized = "FALSE",
            altsyncram_component.read_during_write_mode_port_a =
"NEW_DATA_NO_NBE_READ",
            altsyncram_component.widthad_a = 7,
            altsyncram_component.width_a = 9,
            altsyncram_component.width_byteena_a = 1;

endmodule
```

MIF FILE:

```
WIDTH=9;
DEPTH=128;

ADDRESS_RADIX=HEX;
DATA_RADIX=BIN;

CONTENT BEGIN
      00  :   001001000;
      01  :   001111000;
      02  :   001011000;
      03  :   010000000;
      04  :   001100000;
      05  :   000000001;
      06  :   001010000;
      07  :   101100000;
      08  :   111001010;
      09  :   101001011;
      0A  :   001110000;
      0B  :   000001100;
      0C  :   001101000;
      0D  :   111111111;   //change to 00000010 for waveform stimulation
      0E  :   001010000;
      0F  :   111111111;   //change to 00000010 for waveform stimulation
      10  :   001000000;
      11  :   000010000;
      12  :   011010100;
      13  :   110111000;
      14  :   001000000;
      15  :   000001110;
      16  :   011101100;
      17  :   110111000;
      18  :   001000000;
      19  :   000001100;
      1A  :   011110100;
      1B  :   110111000;
      1C  :   001111000;
      1D  :   000000110;
      [1E..7F]  :   000000000;
END;
```

Laboratory 6:

AN ENHANCED PROCESSOR WITH FLOATING POINT UNIT

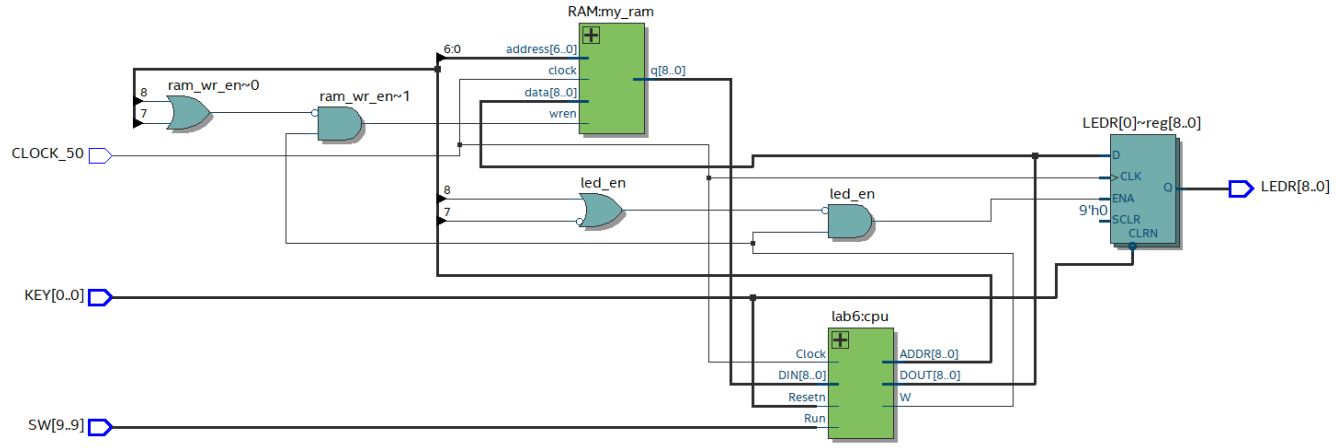
CODE TESTBENCH:

```
`timescale 1ns/1ps
module DE10_tb;
    // 1. Khai báo các tín hiệu kết nối với kit DE10
    logic       CLOCK_50;   // Xung clock 50MHz
    logic [0:0] KEY;        // KEY[0] là Resetn
    logic [9:9] SW;         // SW[9] là Run
    logic [8:0] LEDR;       // Ngõ ra hiển thị LED

    // 2. Gọi module Top-level (DUT - Device Under Test)
    DE10 dut (
        .CLOCK_50(CLOCK_50),
        .KEY(KEY),
        .SW(SW),
        .LEDR(LEDR)
    );

    // 3. Tạo xung Clock 50MHz )
    initial begin
        CLOCK_50 = 0;
        forever #10 CLOCK_50 = ~CLOCK_50;
    end

    // 4. Mô phỏng
    initial begin
        // --- Trạng thái 1: Reset hệ thống ---
        KEY[0] = 0;
        SW[9]  = 0;
        #100;

        // --- Trạng thái 2: Chuẩn bị chạy ---
        KEY[0] = 1;
        #100;

        // --- Trạng thái 3: Kích hoạt Run ---
        SW[9] = 1;

        // Cấp thời gian mô phỏng 5 triệu ns.
        // Đủ để bất kỳ file .mif nào có vòng lặp trễ nhỏ (delay = 2) chạy được
~150+ vòng lặp.
        #500000;

        // Dừng mô phỏng an toàn
        $stop;
    end
    // 5. Theo dõi tín hiệu (Chỉ in ra khi LEDR thay đổi để log console không bị
rác)
    initial begin
        $monitor("[%0t ns] LEDR (Bin): %b | LEDR (Hex): %h", $time, LEDR, LEDR);
    end

endmodule
```

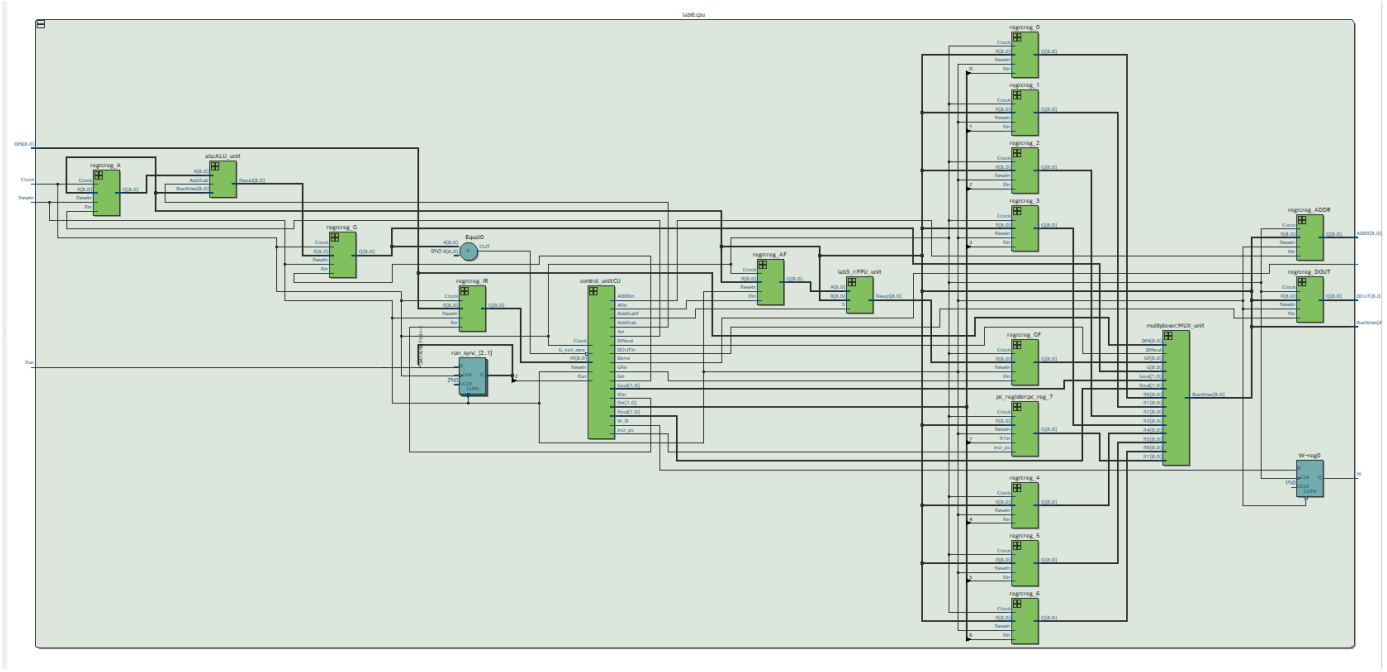
Laboratory 6:

AN ENHANCED PROCESSOR WITH FLOATING POINT UNIT

RTL VIEWER:



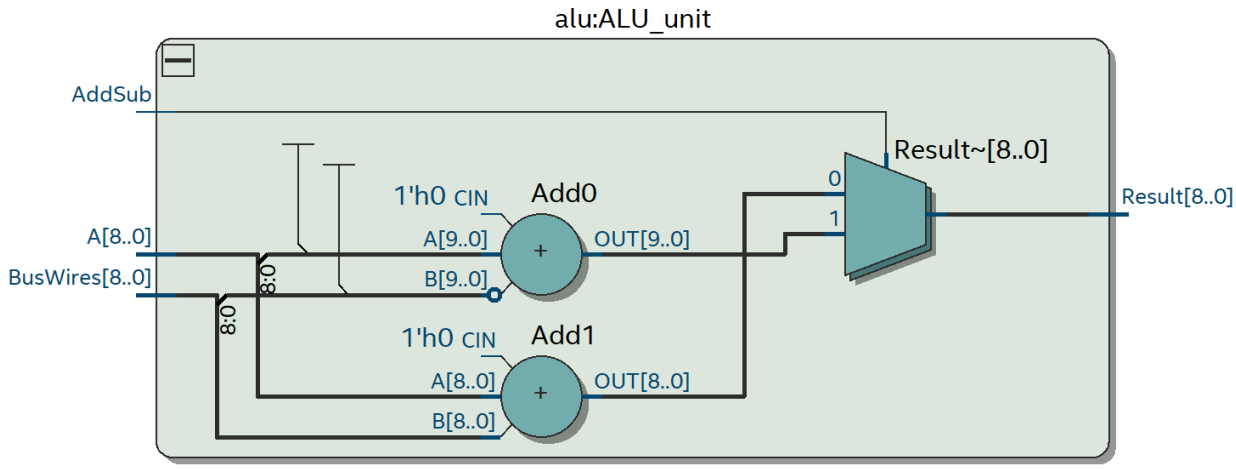
CPU:



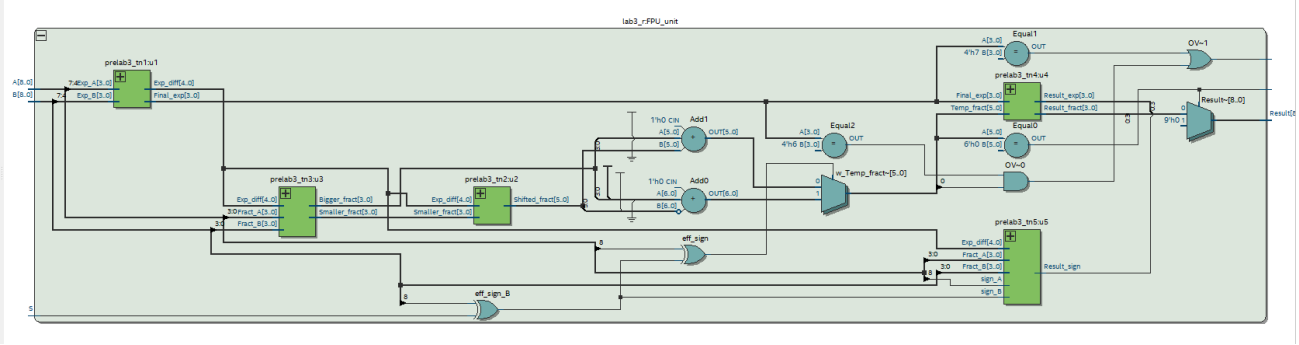
Laboratory 6:

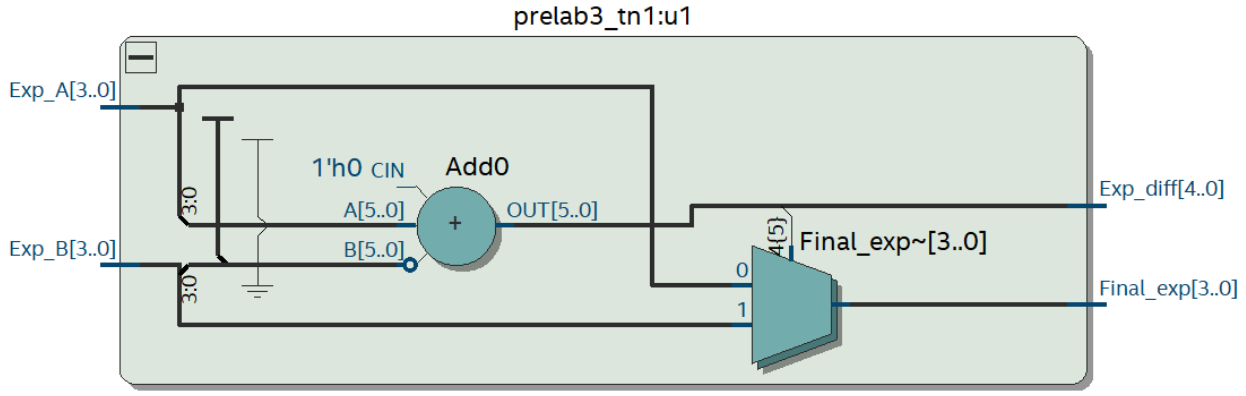
AN ENHANCED PROCESSOR WITH FLOATING POINT UNIT

ALU:



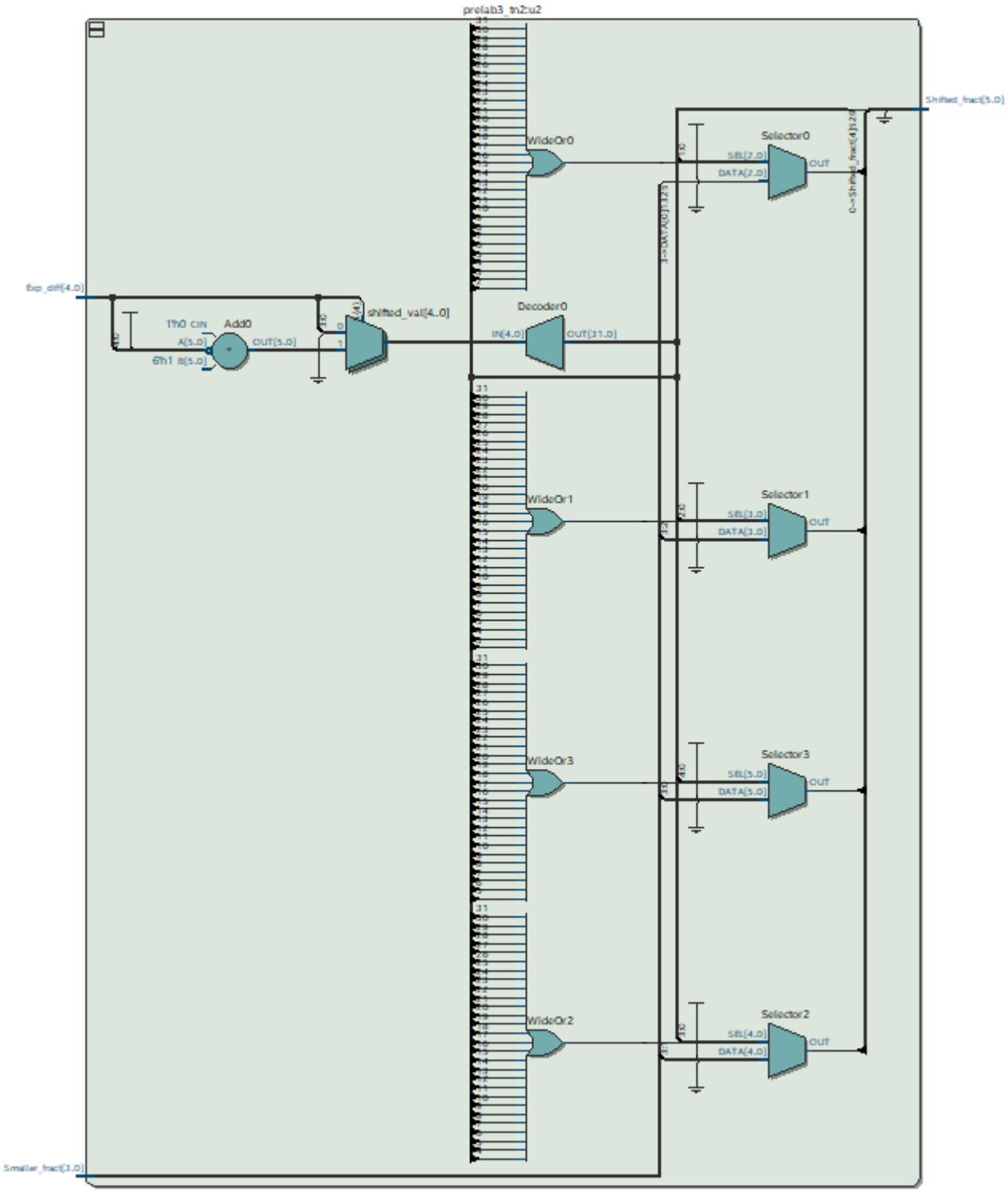
FPU:





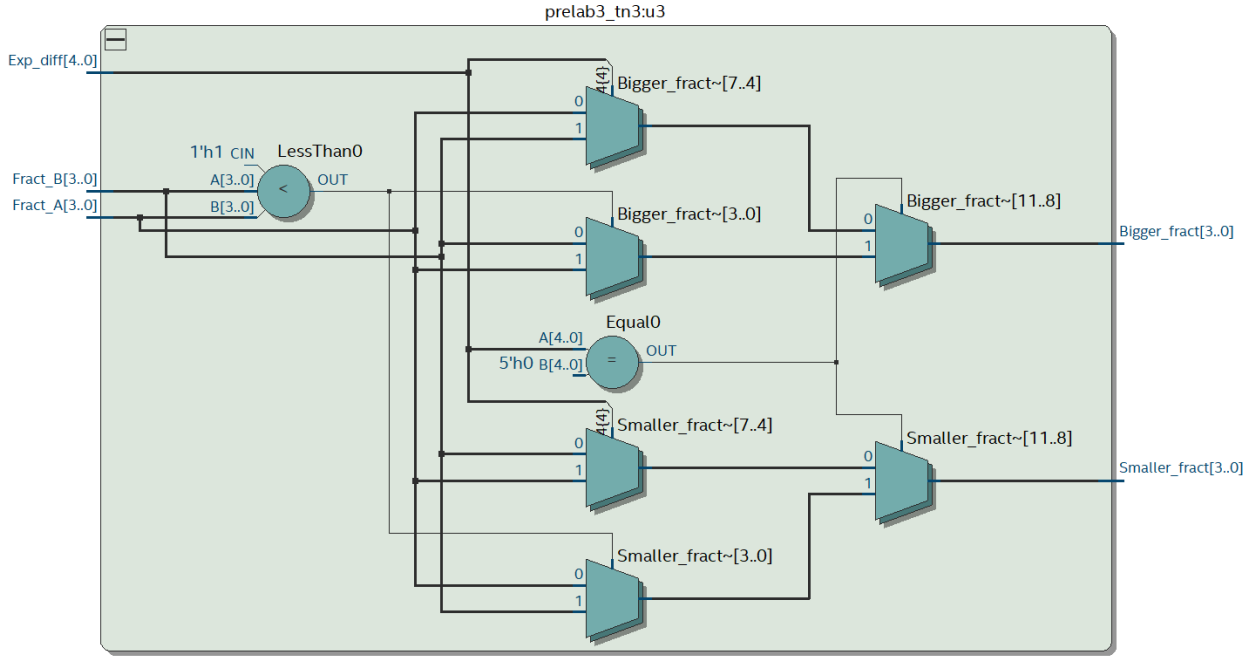
Laboratory 6:

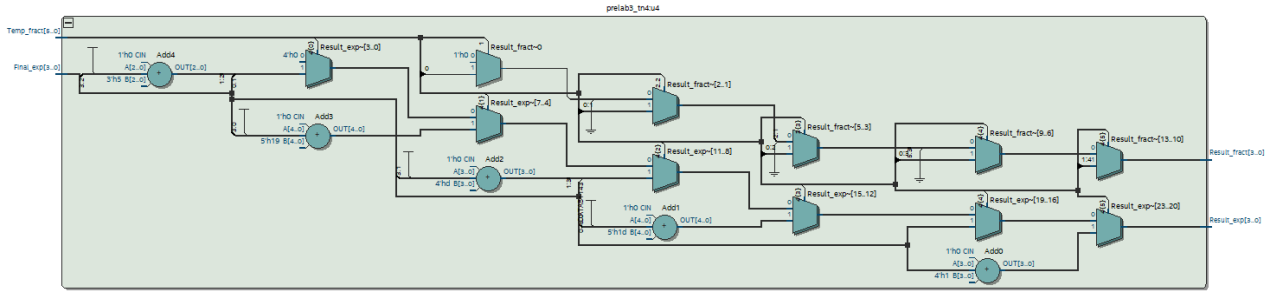
AN ENHANCED PROCESSOR WITH FLOATING POINT UNIT

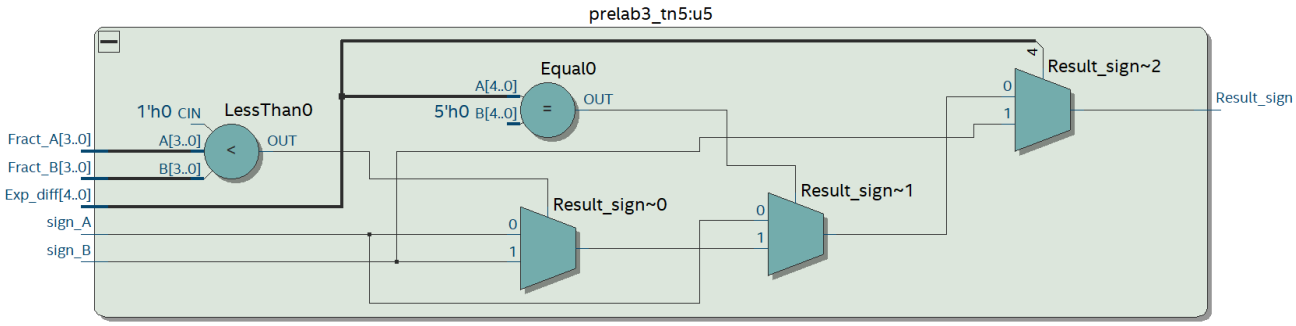


Laboratory 6:

AN ENHANCED PROCESSOR WITH FLOATING POINT UNIT



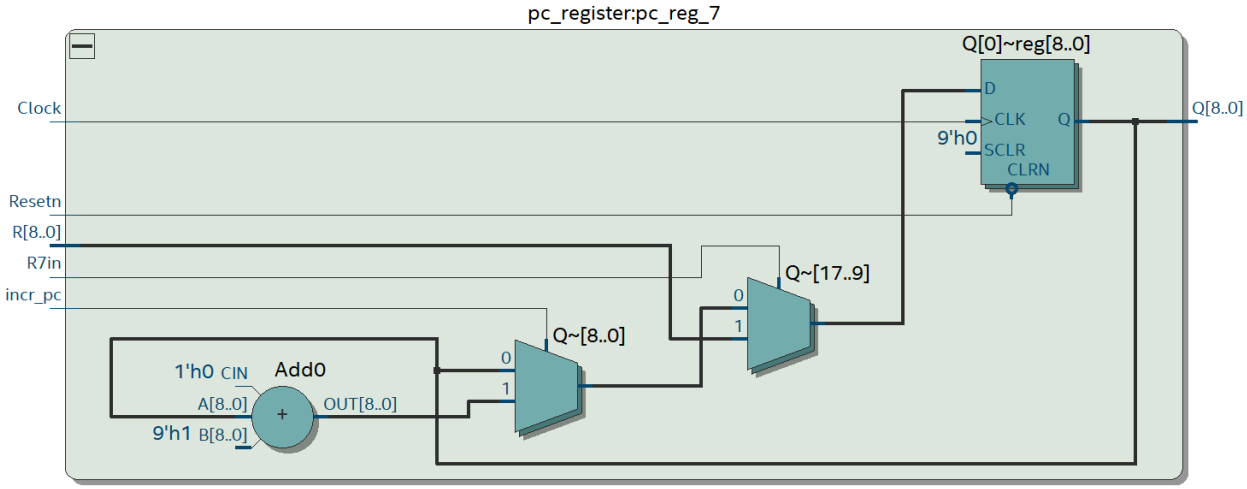




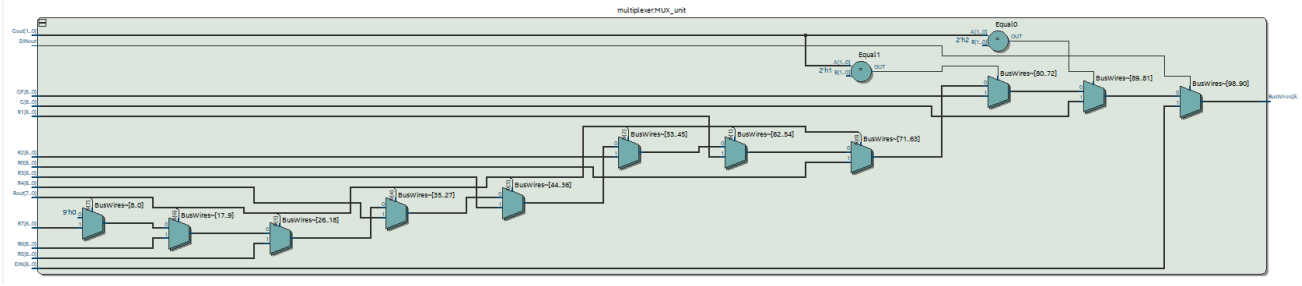
Laboratory 6:

AN ENHANCED PROCESSOR WITH FLOATING POINT UNIT

REGISTER R7:



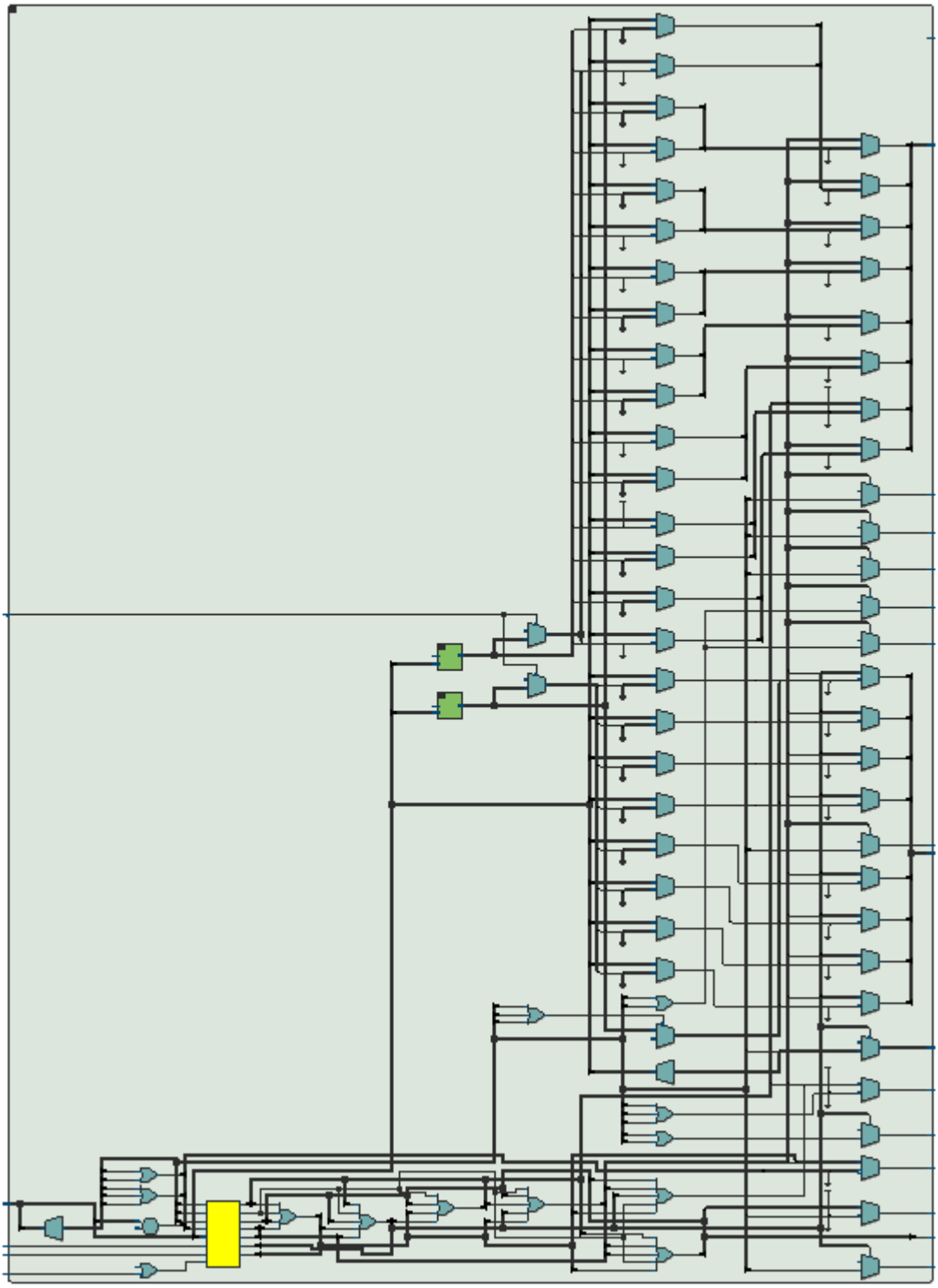
MUX:



Laboratory 6:

AN ENHANCED PROCESSOR WITH FLOATING POINT UNIT

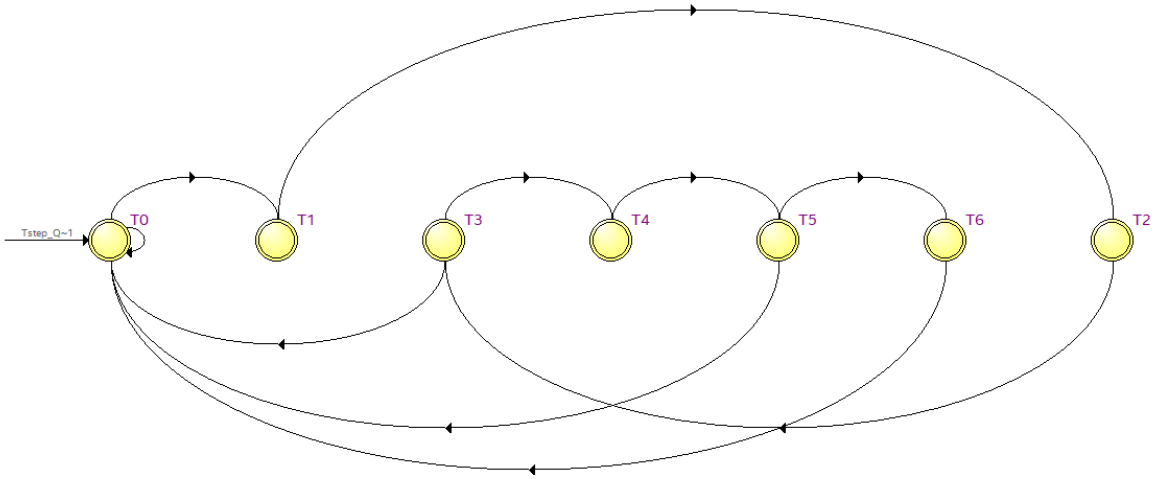
CONTROL UNIT:



Laboratory 6:

AN ENHANCED PROCESSOR WITH FLOATING POINT UNIT

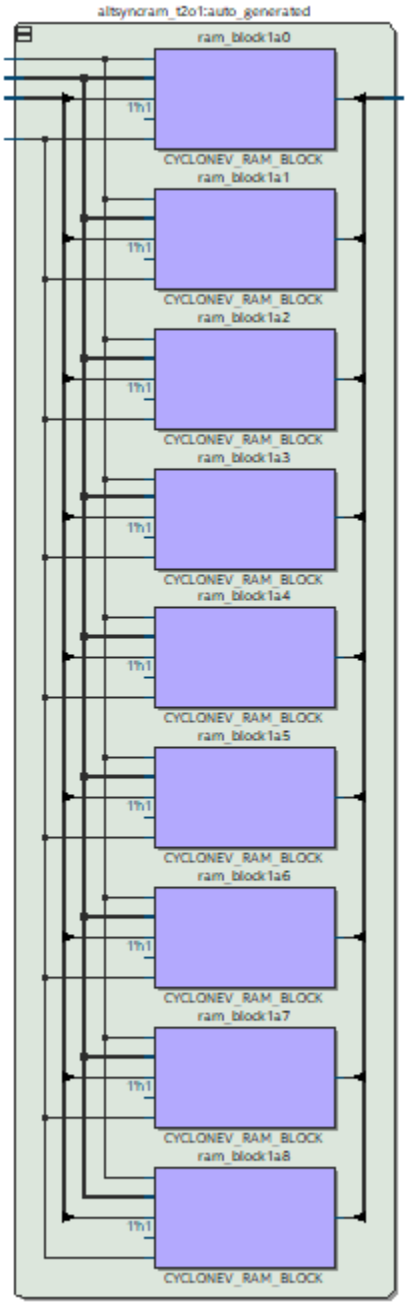
FSM:



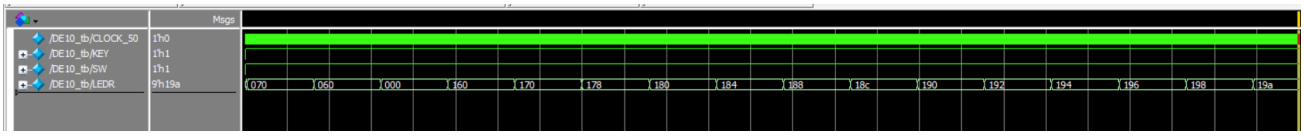
Laboratory 6:

AN ENHANCED PROCESSOR WITH FLOATING POINT UNIT

RAM (IP CATALOG):



WAVEFORM:



We take 1.5 - 0.5 continuously. It will stop at -16 because to align 0.5 (exponent -1) with -16.0 (exponent 4), the hardware right-shifts the 0.5's fraction by their difference of 5 positions. Because this 5-position shift exceeds your 4-bit limit, the 0.5 is pushed completely off the register to zero, leaving -16.0 stuck subtracting nothing.

Hex Value (Waveform)	Binary (S - EEEE - FFFF) – LED	Mathematical Equation	Decimal Value
070	0 0111 0000	$+1.0000_2 \times 2^{(7-7)}$	1.0
060	0 0110 0000	$+1.0000_2 \times 2^{(6-7)}$	0.5
000	0 0000 0000	*Special Case (Zero)*	0.0
160	1 0110 0000	$-1.0000_2 \times 2^{(6-7)}$	-0.5
170	1 0111 0000	$-1.0000_2 \times 2^{(7-7)}$	-1.0
178	1 0111 1000	$-1.1000_2 \times 2^{(7-7)}$	-1.5
180	1 1000 0000	$-1.0000_2 \times 2^{(8-7)}$	-2.0
184	1 1000 0100	$-1.0100_2 \times 2^{(8-7)}$	-2.5
188	1 1000 1000	$-1.1000_2 \times 2^{(8-7)}$	-3.0
18c	1 1000 1100	$-1.1100_2 \times 2^{(8-7)}$	-3.5
190	1 1001 0000	$-1.0000_2 \times 2^{(9-7)}$	-4.0
192	1 1001 0010	$-1.0010_2 \times 2^{(9-7)}$	-4.5
194	1 1001 0100	$-1.0100_2 \times 2^{(9-7)}$	-5.0

Laboratory 6:

AN ENHANCED PROCESSOR WITH FLOATING POINT UNIT

Hex Value (Waveform)	Binary (S - EEEE - FFFF) – LED	Mathematical Equation	Decimal Value
196	1 1001 0110	$-1.0110_2 \times 2^{(9-7)}$	-5.5
198	1 1001 1000	$-1.1000_2 \times 2^{(9-7)}$	-6.0
19a	1 1001 1010	$-1.1010_2 \times 2^{(9-7)}$	-6.5

STIMULATION: It will take 1,5 – 0.5 continuously

1.5 – 0.5 = 1

1 – 0.5 = 0.5

Laboratory 6:

AN ENHANCED PROCESSOR WITH FLOATING POINT UNIT

0.5 – 0.5 = 0

0 – 0.5 = -0.5

Laboratory 6:

AN ENHANCED PROCESSOR WITH FLOATING POINT UNIT

It will minus 0.5 continuously, until -16 just like the waveform.

LINK TO THE STIMULATION VIDEO:
https://drive.google.com/file/d/1osukKjodTKgUkE9WIVXgjY3MJ7c4g7Sw/view?usp=drive_link